

ISIS 3710 — Programación con Tecnologías Web

Examen Parcial — Módulo Backend

Práctico · 15 de mayo de 2026

1. Contexto

La Biblioteca Central de la universidad necesita un sistema para gestionar préstamos de libros y equipos audiovisuales a sus miembros. Cada miembro puede tener hasta cierto número de préstamos activos a la vez; cada item solo puede estar prestado a un miembro a la vez; los préstamos vencidos generan multas calculadas por día de retraso.

Usted recibe un repositorio base ligero y debe construir el sistema completo en el tiempo del examen.

Reglas generales

- Trabajo **individual**. No se permite colaboración entre estudiantes.
- Sí se permite consultar: documentación oficial (NestJS, TypeORM, class-validator, MDN), Stack Overflow, y el proyecto **MediTrack** (<https://github.com/wareval0/MediTrack-API>) como referencia de patrones.
- Evaluación 100% sobre el código entregado. Verifique antes de subir que arranca con `npm install && npm run migration:run && npm run start:dev`.

2. Dominio y conceptos

Actores

Rol	Función
admin	Administra el sistema completo.
librarian	Crea/edita items, registra préstamos y devoluciones.
member	Consulta items, ve sus propios préstamos, recibe multas.

Entidades del dominio

- **Item**: un libro o equipo prestable. Tiene un `code` único (ej. BK-0042, EQ-LAB-007), título, y tipo (book, magazine, equipment).
- **Loan (préstamo)**: un miembro toma prestado un item por una ventana [`loanedAt`, `dueAt`]. Si devuelve a tiempo, paga 0. Si devuelve tarde, paga multa por cada día de retraso.

3. Repositorio base entregado (scaffolding ligero)

El repo base incluye solo infraestructura; usted debe construir lo demás. Incluye:

- **docker-compose.yml** con postgres:16-alpine. Arranca con `docker compose up -d`.
- **package.json** con scripts: `start:dev`, `test`, `test:e2e`, `migration:generate`, `migration:run`, `lint`, y dependencias preinstaladas: `@nestjs/*`, `typeorm`, `pg`, `class-validator`, `class-transformer`, `joi`, `bcrypt`, `@nestjs/jwt`, `@nestjs/passport`, `passport-jwt`, `@nestjs/swagger`, `jest`, `supertest`.
- **src/main.ts** con:
 - `ValidationPipe` global (`whitelist`, `forbidNonWhitelisted`, `transform`).
 - Prefijo global `/api`.
 - Swagger UI montado en `/api/docs`.
- **src/config/** con `ConfigModule` + validación `Joi` (variables requeridas: `DB_*`, `JWT_ACCESS_SECRET`, etc.).
- **src/database/data-source.ts** configurado para la CLI de TypeORM. **Sin migraciones todavía.**
- **.env.example** con valores listos para `cp .env.example .env`.
- **README mínimo** con los comandos de arranque.

Lo que **NO** viene en el scaffolding y debe implementar:

- Entidad `User` y módulo `auth` completo.
- Entidades `Item` y `Loan`.
- Migraciones.
- Tests.

4. Parte Obligatoria (100%)

4.1. Autenticación básica (20%)

Implemente el módulo `auth`:

Entidad User (tabla users)

Campo	Tipo	Notas
<code>id</code>	<code>uuid</code>	PK, <code>@PrimaryGeneratedColumn('uuid')</code>
<code>email</code>	<code>varchar(255)</code>	único, indexado
<code>passwordHash</code>	<code>varchar(255)</code>	hash <code>bcrypt</code> — nunca guardar el plain
<code>firstName</code>	<code>varchar(100)</code>	requerido
<code>lastName</code>	<code>varchar(100)</code>	requerido

Campo	Tipo	Notas
role	enum	admin librarian member, default member
isActive	boolean	default true
createdAt, updatedAt	timestamp	

Endpoints

Método	Ruta	Función
POST	/auth/register	Crear User con rol member. Recibe { email, password, firstName, lastName }. Devuelve { accessToken, user }.
POST	/auth/login	Validar credenciales. Devuelve { accessToken, user }.
GET	/auth/me	Requiere JWT. Devuelve el usuario autenticado.

Requisitos

- Hash de password con **bcrypt** (saltRounds configurable, default 10). **No guardar plain.**
- Access token JWT firmado con JWT_ACCESS_SECRET (mínimo 32 chars). Expira en 15m (configurable).
- Payload del JWT: { sub: user.id, email, role }.
- JwtStrategy que extraiga el bearer del header Authorization.
- JwtAuthGuard aplicado **globalmente**; los endpoints públicos (register, login) marcados con @Public().
- DTOs RegisterDto y LoginDto con class-validator (@IsEmail, @MinLength(8), @IsNotEmpty).
- **Nunca devolver passwordHash** en respuestas (use @Exclude o un transformer).

No se exige refresh tokens en obligatorio. Eso es bono B2.

4.2. Modelado de datos: Item y Loan (15%)

Item (tabla items)

Campo	Tipo	Notas
id	uuid	PK
code	varchar(32)	único, indexado (ej. BK-0042)
title	varchar(255)	requerido
type	enum	book magazine equipment

Campo	Tipo	Notas
isActive	boolean	default true (soft delete)
createdAt, updatedAt	timestamp	

Loan (tabla loans)

Campo	Tipo	Notas
id	uuid	PK
userId	uuid	FK → User(id), ON DELETE RESTRICT
itemId	uuid	FK → Item(id), ON DELETE RESTRICT
loanedAt	timestampz	requerido
dueAt	timestampz	requerido. Debe ser > loanedAt
returnedAt	timestampz	nullable
status	enum	active returned overdue lost, default active
fineAmount	decimal(10,2)	default 0.00
createdAt, updatedAt	timestamp	

Índices y restricciones

- Índice compuesto sobre (itemId, status) — usado para verificar disponibilidad.
- Índice sobre (userId, status) — usado para contar préstamos activos por usuario.
- No es necesario poner UNIQUE PARTIAL (en Postgres WHERE status = 'active') — la regla de “un item solo puede estar prestado a uno a la vez” se valida en el service.

Migración

Se recomienda crear una migración TypeORM que produzca este schema (incluyendo la tabla users de la sección 4.1). Evite synchronize: true en ambientes de producción.

4.3. Servicios, controladores y DTOs (25%)

Implemente ItemsModule y LoansModule, **todos autenticados** (sin @Public()):

ItemsModule

Método	Ruta	Función
POST	/items	Crear item. Recibe { code, title, type }.
GET	/items	Listar items activos. Filtro opcional ?type=book magazine equipment. Cada item debe incluir isAvailable: boolean (calculado: no tiene préstamo activo).
GET	/items/:id	Detalle de un item, incluido isAvailable.
PATCH	/items/:id	Actualizar title o type.
DELETE	/items/:id	Soft delete (isActive = false).

LoansModule

Método	Ruta	Función
POST	/loans	Crear préstamo. Recibe { userId, itemId, dueAt }. Aplica reglas de la sección 4.4. loanedAt = now() lo asigna el servidor.
GET	/loans	Listar préstamos. Filtros opcionales: ?userId=&itemId=&status=.
GET	/loans/:id	Detalle de un préstamo.
PATCH	/loans/:id/return	Marcar como devuelto. Calcula multa según sección 4.4. No recibe body (server-side calculation).
PATCH	/loans/:id/mark-lost	Marcar como lost.

Requisitos transversales

- DTOs con class-validator (@IsUUID, @IsDateString, @IsEnum, @IsString, @MaxLength, @IsOptional).
- DTOs con @ApiProperty para Swagger.
- Controllers protegidos con @ApiBearerAuth().
- Códigos HTTP correctos: 201 POST, 200 GET/PATCH, 204 DELETE, 404 recurso inexistente, 400 DTO inválido, 409 conflicto de negocio.

4.4. Reglas de negocio críticas (25%)

El LoansService debe aplicar las siguientes reglas:

Regla R1 — Validación de fechas

Al crear un préstamo:

- dueAt > loanedAt (donde loanedAt = now() en el servidor).
- dueAt - loanedAt ≤ 30 días (ventana máxima de préstamo).

Si no cumple: lance `BadRequestException` (HTTP 400).

Regla R2 — Item disponible

No se puede crear un préstamo de un item que ya tiene un préstamo con `status ∈ {active, overdue}`. Si lo intenta: lance `ConflictException` (HTTP 409) con mensaje claro indicando el `loanId` que bloquea.

Regla R3 — Límite de préstamos simultáneos por usuario

Un usuario no puede tener más de `MAX_ACTIVE_LOANS_PER_USER = 3` préstamos con `status ∈ {active, overdue}` a la vez. La constante puede vivir en el config (`process.env.MAX_ACTIVE_LOANS`, default 3). Si intenta tomar un cuarto: lance `ConflictException` (HTTP 409).

Regla R4 — Cálculo de multa al devolver

En PATCH `/loans/:id/return`:

```
returnedAt = now()
daysOverdue = max(0, ceil((returnedAt - dueAt) / 1 día))
fineAmount = daysOverdue × DAILY_FINE_RATE
status = 'returned' (sin importar si hubo retraso)
```

donde `DAILY_FINE_RATE = 0.50` USD/día (configurable vía `process.env.DAILY_FINE_RATE`).

Ejemplos numéricos para verificar:

dueAt	returnedAt	daysOverdue	fineAmount
2026-01-10	2026-01-10	0	0.00
2026-01-10	2026-01-11	1	0.50
2026-01-10	2026-01-15	5	2.50
2026-01-10	2026-01-12 12:00 (2.5 días reales)	3 (ceil)	1.50

Nota: use `Math.ceil` sobre días, no días enteros. Devolver 1 minuto tarde cuenta como 1 día de multa.

Regla R5 — Transiciones FSM de Loan

Desde / Hacia	active	overdue	returned	lost
active	—	true (automático por tiempo)	true (al devolver)	true (al marcar perdido)
overdue	—	—	true (al devolver tarde)	true (al marcar perdido)
returned	—	—	—	— (terminal)
lost	—	—	—	— (terminal)

- `returned` y `lost` son **terminales**: si alguien intenta `return` sobre un loan ya `returned` o `lost`, lance `BadRequestException` (HTTP 400).

- La transición a overdue puede ser automática vía query (no se exige un job cron — basta con que `GET /loans?status=overdue` filtre los préstamos donde `dueAt < now() AND status = 'active' AND returnedAt IS NULL`). Opcionalmente, puede actualizar el status en BD al consultarlo. Documente su decisión en el README.

4.5. Tests unitarios + Swagger + README (15%)

Tests unitarios (10%)

Escriba **al menos 4 tests unitarios** del `LoansService` con el mock del repositorio. Casos mínimos:

- Crea préstamo exitoso cuando item disponible, usuario bajo el límite y fechas válidas.
- Lanza `ConflictException` si el item ya tiene un préstamo activo (R2).
- Lanza `ConflictException` si el usuario ya tiene 3 préstamos activos (R3).
- `return` calcula multa correctamente: dado un préstamo con `dueAt = hace 5 días`, al devolver hoy debe calcular `fineAmount = 5 × 0.50 = 2.50` (R4).

No use base de datos real para los tests unitarios.

Swagger (3%)

- Swagger UI funcional en `http://localhost:3000/api/docs` con todos los endpoints, DTOs tipados, `@ApiBearerAuth()` en los protegidos.

README (2%)

- Comandos de arranque: `cp .env.example .env`, `docker compose up -d`, `npm install`, `npm run migration:run`, `npm run start:dev`.
- Credenciales de prueba (un usuario admin seed si lo implementa, o instrucciones).
- Decisión sobre la transición automática a `overdue` (4.4 R5).
- Lista de bonos implementados (si los hay).

5. Bonos

Bono B1 — Cola FIFO de reservas — +8%

Cuando un item está prestado, otros miembros pueden reservarlo y entrar en una cola. Al devolverse el item, el primero de la cola tiene preferencia.

Nueva entidad `Reservation` (tabla `reservations`)

Campo	Tipo	Notas
<code>id</code>	<code>uuid</code>	PK
<code>userId</code>	<code>uuid</code>	FK → <code>User(id)</code>

Campo	Tipo	Notas
itemId	uuid	FK → Item(id)
createdAt	timestampz	usado para ordenar la cola (FIFO)
fulfilledAt	timestampz	nullable. Se llena cuando el item se le asigna.
cancelledAt	timestampz	nullable. Si el usuario cancela su reserva.
expiresAt	timestampz	la reserva expira 48 horas después de notificarse al usuario.

Endpoints nuevos

Método	Ruta	Función
POST	/reservations	Crear reserva. Recibe { itemId }. Falla si el item está disponible.
GET	/reservations	Listar reservas del usuario actual + filtros admin/librarian.
DELETE	/reservations/:id	Cancelar reserva propia (set cancelledAt = now()).

Reglas críticas del bono

- **R-B1.1:** Un usuario no puede tener más de 1 reserva pendiente para el mismo item.
- **R-B1.2:** Al devolver un loan (PATCH /loans/:id/return), busque la primera reserva pendiente del mismo itemId ordenada por createdAt ASC. Si existe, márkela como fulfilledAt = now() y expiresAt = now() + 48h.
- **R-B1.3:** Si un usuario con reserva fulfilledAt no toma el préstamo antes de expiresAt, su reserva expira; al siguiente return del item, la cola salta a la siguiente.
- **R-B1.4:** Al crear un loan, si hay reservas pendientes para ese item, **solo el primero de la cola puede tomarlo**. Otros usuarios reciben ForbiddenException (HTTP 403) con mensaje claro.

Verificación esperada

```
member1 toma loan de itemA
member2 hace reservation de itemA → 201
member3 hace reservation de itemA → 201
member1 devuelve itemA
  → reservation de member2 pasa a fulfilledAt = now()
member3 intenta POST /loans { itemId: itemA } → 403
member2 hace POST /loans { itemId: itemA } → 201
```

Bono B2 — Refresh tokens stateful — +5%

Implemente el flujo completo de refresh tokens persistidos en BD:

- Entidad RefreshToken (tabla refresh_tokens): id, userId (FK), token (unique), expiresAt, revokedAt (nullable), createdAt.

- Dos secretos distintos: `JWT_ACCESS_SECRET` y `JWT_REFRESH_SECRET` (ambos ≥ 32 chars, validados por Joi).
- Access expira en 15m; refresh expira en 7d.
- `POST /auth/login` y `POST /auth/register` devuelven `{ accessToken, refreshToken, user }` y persisten el refresh.
- `POST /auth/refresh` (público): valida firma del refresh + lookup en BD (existe, no `revokedAt`, no `expiresAt < now()`). Si pasa: emite nuevo `accessToken`.
- `POST /auth/logout` (autenticado): recibe `{ refreshToken }`, marca `revokedAt = now()`. Después de logout, ese refresh debe fallar con 403.

Referencia: `auth.service.ts` en MediTrack (<https://github.com/wareval0/MediTrack-API>). **No copie:** la firma del payload, el manejo de `jwt`, y la integración con el endpoint son ejercicios suyos.

Bono B3 — Pipeline GitHub Actions — +4%

Cree `.github/workflows/ci.yml` que en push a cualquier rama y en pull request a `main` ejecute:

- Checkout.
- Setup Node 20.
- `npm ci`.
- `npm run lint`.
- `npm test`.

Debe usar un **servicio de Postgres efímero** (`services: postgres:`) si sus tests lo requieren. El job debe fallar si cualquier paso falla.

+1% adicional si además construye la imagen Docker con `docker build` (sin necesidad de publicarla).

Referencia: `.github/workflows/ci-cd.yml` en MediTrack (<https://github.com/wareval0/MediTrack-API>).

Bono B4 — Tests e2e + matriz FSM — +3%

Dos sub-entregables:

- **Suite e2e con supertest** que cubra el flujo: `register` → `login` → `POST /items` → `POST /loans` → `PATCH /loans/:id/return`. Verifique que la respuesta del return incluye `fineAmount`.
- **Tests parametrizados de la matriz FSM de Loan** con `it.each` cubriendo al menos:
 - 2 transiciones válidas (ej. `active` → `returned`, `active` → `lost`).
 - 3 transiciones inválidas (ej. `returned` → `active`, `lost` → `returned`, `returned` → `lost`) que devuelven HTTP 400.

Referencia: la sección “matriz de transiciones FSM” en `appointments.e2e-spec.ts` de MediTrack (<https://github.com/wareval0/MediTrack-API>) muestra el patrón con `it.each`.

6. Restricciones técnicas (penalización parcial)

- **No desactive el `JwtAuthGuard` global** ni la `ValidationPipe` global “para que pase”.
 - **No `hardcodee` secretos** ni credenciales. Todo en `.env`. El archivo `.env` no va al repo (debe estar en `.gitignore`).
 - **No guarde passwords en texto plano.** Use `bcrypt`.
 - **(Recomendación) No use `synchronize: true`** como mecanismo de creación de schema en la rama de evaluación. Use migración.
-

7. Entrega y evaluación

Cómo entregar

- Suba la URL de un repo Git al que el profesor y monitor tengan acceso.

Método de evaluación

Asegúrese de que los siguientes pasos funcionen:

- `cp .env.example .env`
 - `docker compose up -d`
 - `npm install`
 - `npm run migration:run`
 - `npm run start:dev`
 - Verificar Swagger en `/api/docs`.
 - **Flujo de validación:**
 - `POST /auth/register` un admin/librarian + 2 members.
 - `POST /auth/login` y captura del token.
 - `POST /items` con `{ code: "BK-001", title: "El Quijote", type: "book" }`.
 - `POST /loans` con `{ userId, itemId, dueAt: now + 7d }` → 201.
 - `POST /loans` repetido para el mismo `itemId` → 409 Conflict (R2).
 - `POST /loans` con 4 préstamos consecutivos para el mismo usuario → al 4to 409 Conflict (R3).
 - `POST /loans` con `dueAt < loanedAt` → 400 Bad Request (R1).
 - `PATCH /loans/:id/return` después de simular retraso → verificar `fineAmount` calculado correcto (R4).
 - `PATCH /loans/:id/return` sobre un loan ya returned → 400 Bad Request (R5).
 - `npm test` debe pasar.
 - Validación de bonos declarados en el README.
-

8. Rúbrica

Sección	Criterio	Peso
4.1	Auth: User + register + login + JWT + guards + bcrypt	20%
4.2	Modelado Item y Loan + migración funcional	15%
4.3	Servicios, controllers, DTOs con class-validator + Swagger	25%
4.4	Reglas R1–R5 (validación, disponibilidad, límite, multa, FSM)	25%
4.5	4 tests unitarios + Swagger + README	15%
	Total	100%

Bonos

Bono	Criterio	Extra
B1	Cola FIFO de reservas (Reservation + 4 reglas)	+8%
B2	Refresh tokens stateful	+5%
B3	GitHub Actions CI (con +1% por build Docker)	+4%
B4	Suite e2e + matriz FSM parametrizada	+3%
	Máximo de bonos	+20%

Nota final = mín(120%, obligatorio + bonos).

9. Consejos finales

- **Empiece por auth y migraciones.** Sin auth, no puede probar nada. Sin migraciones, no puede levantar la BD.
- **Pruebe Swagger o Postman temprano.** Si `/api/docs` no carga, los siguientes pasos son a ciegas.
- **Implemente la regla R4 (multa) con tests primero.** Es la que más fácilmente se equivoca con `Math.floor` vs `Math.ceil` o con zonas horarias.
- **Para el bono B1 (cola):** piense bien el orden de operaciones. Al hacer `return`, el orden es: marcar loan returned → buscar primera reservation pendiente → asignarla. No pasar por encima.
- **Antes de entregar:** corra `npm run lint`, `npm test`, y arranque desde cero (`docker compose down -v && up -d && migration:run && start:dev`) para asegurar que su `.env.example` es suficiente.

¡Éxitos!